

Model-Based Continuous-Time Policy Evaluation with Unknown Lévy Process Dynamics

Research Report from July to October

Lin Wang¹

¹Undergraduate of Zhejiang University
Department of Mathematics

18, October 2024

Personal Introduction



Name: Lin Wang (王麟)

Education: Zhejiang University

Major: Mathematics and Applied Mathematics

GPA: 3.96/4.0 (Ranked 1st)

Interests: Statistic Learning Theory, Numerical Optimization, Functional Analysis

Email: wanglin1126@zju.edu.cn

Research Experience

- Prediction of Physical Properties of Crystals Based on Deep Graph Neural Networks
- Demonstration of Multi-factors and GRU Time Series Models
- A Model-Based Approach for Continuous-Time Policy Evaluation with Unknown Lévy Process Dynamics

Task Breakdown

1 Parameter Recovery

- Overview
- Diagonal Diffusion Coefficient
- Positive Defined Diffusion Coefficient

2 Solving PDEs with PINNs

- One-dimensional Burgers Equation
- 2D nonlinear Navier-Stokes equation

Task Breakdown

1 Parameter Recovery

- Overview
- Diagonal Diffusion Coefficient
- Positive Defined Diffusion Coefficient

2 Solving PDEs with PINNs

- One-dimensional Burgers Equation
- 2D nonlinear Navier-Stokes equation

Parameter Recovery Overview

We formulate the problem as a stochastic optimal control problem as follows:

$$\begin{cases} V(x) = \mathbb{E}_\pi \left[\int_0^\infty r(X_t, a_t) e^{-\gamma t} dt \middle| X(t_0) = x \right], \\ dX_t = b(X_t, a_t)dt + \sigma(X_t, a_t)dW_t. \end{cases} \quad (1)$$

The corresponding value function satisfies:

$$\beta[V](x) = r(x) + b(x) \left[\frac{\partial}{\partial x} V \right](x) + D_o(x) \left[\frac{\partial^2}{\partial s^2} V \right](x) - D_f(x) [(-\Delta)^\alpha V](x)$$

$$[(-\Delta)^\alpha f](x) = [\mathcal{F}^{-1}(|\xi|^{2\alpha}[\mathcal{F}(f)](\xi))](x),$$

where $\mathcal{F}$ represents the Fourier transform.

Diagonal Experiment Without Jump

Firstly, we try to recovery the parameters of

$$d\mathbf{X}(t) = \mathbf{b}(\mathbf{X}(t), t)dt + \boldsymbol{\sigma}(\mathbf{X}(t), t)d\mathbf{W}_t, \mathbf{X}(t=0) = \mathbf{X}_0, t \in [0, T].$$

Where $\boldsymbol{\sigma}(\mathbf{X}(t), t)$ is a diagonal matrix in this section.

With the following **Relative Error Criterion**, where $\mathbf{D} = \boldsymbol{\sigma}\boldsymbol{\sigma}^T/2$

$$\begin{aligned} b_{\text{error}} &= \frac{\sum_{i=1}^I \sum_{j=1}^J \|\mathbf{b}(x_{i,j}, t_j) - \hat{\mathbf{b}}(x_{i,j}, t_j)\|_2^2}{\sum_{i=1}^I \sum_{j=1}^J \|\hat{\mathbf{b}}(x_{i,j}, t_j)\|_2^2}, \\ D_{\text{error}} &= \frac{\sum_{i=1}^I \sum_{j=1}^J \|\boldsymbol{\sigma}\boldsymbol{\sigma}^T(x_{i,j}, t_j) - \hat{\boldsymbol{\sigma}}\hat{\boldsymbol{\sigma}}^T(x_{i,j}, t_j)\|_F^2}{\sum_{i=1}^I \sum_{j=1}^J \|\hat{\boldsymbol{\sigma}}\hat{\boldsymbol{\sigma}}^T(x_{i,j}, t_j)\|_F^2}. \end{aligned} \quad (2)$$

$x_{i,j}$ represents the position with index j on the observed trajectory with index i .

Framework

We followed two methods to generate trajectories with the framework of torchsde.

Euler-Maruyama:

$$X_{t+\Delta t} = X_t + f(t, X_t)\Delta t + g(t, X_t)\Delta W_t,$$

Milstein:

$$X_{t+\Delta t} = X_t + f(t, X_t)\Delta t + g(t, X_t)\Delta W_t + \frac{1}{2}g'(t, X_t)((\Delta W_t)^2 - \Delta t).$$

Loss functions includes:

Distance-Based: MSE, MMD, Wasserstein-2Distance(W2) and Mean2+Variance.

Density Function-Based: MLE.

1D SDE

When $\dim b, \sigma = 1$, Let $b(X_t, t) = 5 - X_t$, $\sigma(X_t, t) = 0.5X_t$, $X_0 = 2$

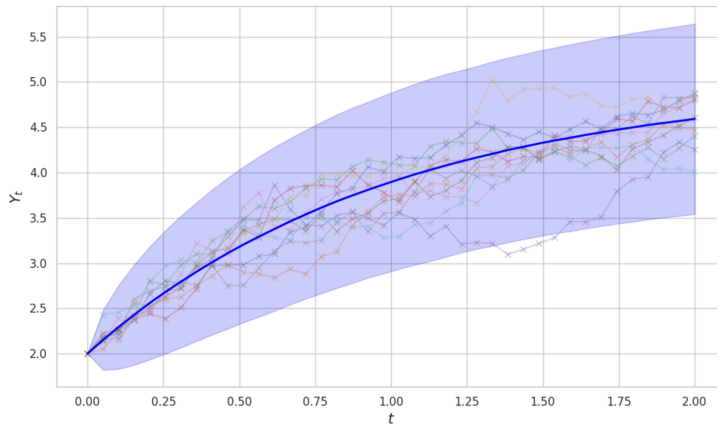
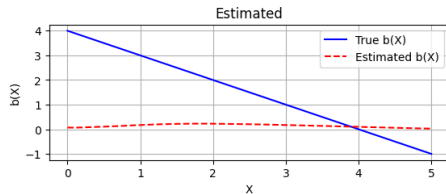
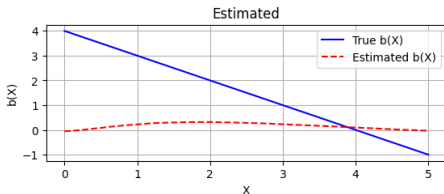
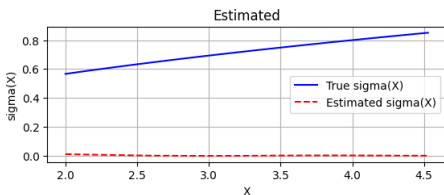


Figure: Trajectory examples, the shadows are drawn according to $\pm 3\sigma$

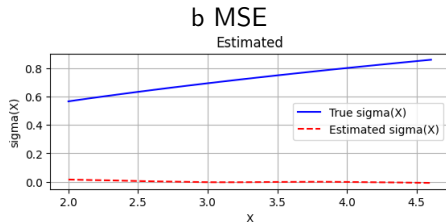
Inefficient Loss



b MMD

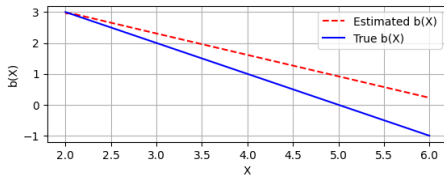
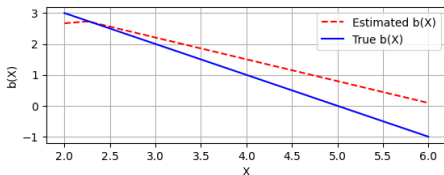


σ MMD

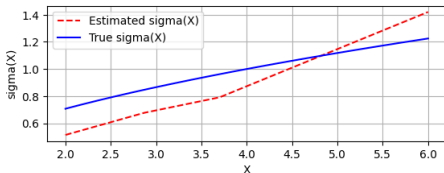


σ MSE

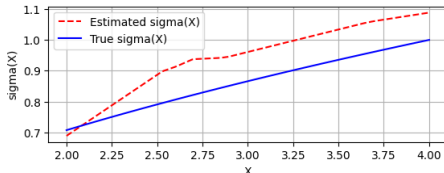
Efficient Loss



b W2 Loss and error 0.0038

 σ W2 Loss and error 0.005

b MLE Loss and error 0.0032

 σ MLE Loss and error 0.007

Comparison Experiment

$b(X_t, t) = 5 - X_t, \sigma(X_t, t) = 0.5X_t, X_0 = 2$				
d	I	b_{error}	D_{error}	b / D converge epochs
1	300	0.0032	0.007	700/100
2	300	0.0185	0.00186	1000/500
5	300	0.0363	0.0017	2000/500
10	300	0.0538	0.001569	2000/1000
20	500	0.03	0.0015	2000
20	1000	0.016	0.0010	10000/3000
50	300	0.139	0.0279	5500/5000
50	1500	0.0543	0.0183	10000/3000
100	300	0.35	0.061	4000
100	1500	0.22	0.026	15000/4000
100	5000	0.052	0.0051	15000/4000

Diagonal Experiment

$b(\mathbf{X}_t, t) = 5 - \mathbf{X}_t, \sigma(\mathbf{X}_t, t) = 0.5\mathbf{X}_t$, different $\mathbf{X}_0$				
d	I	b_{error}	D_{error}	b / D converge epochs
5	900	0.033	0.0397	3000
10	2000	0.0338	0.0298	6000
20	6000	0.154	0.104	10000
$b(\mathbf{X}_t, t) = 5 - \mathbf{X}_t, \sigma(\mathbf{X}_t, t) = 0.5\mathbf{X}_t, \mathbf{X}_0 = 2$				
d	I	b_{error}	D_{error}	b / D converge epochs
5	300	0.0363	0.0017	2000/500
10	300	0.0538	0.001569	2000/1000
20	500	0.03	0.0015	2000

Table: This table shows how relative error changes with different initialization when the dimension increases. In the progress, we need intermittently add new data to the train dataset When a bottleneck is encountered during training.

Complicate Function Experiment

$$b(\mathbf{X}_t, t) = \exp(\cos(\mathbf{X}_t)), \sigma(\mathbf{X}_t, t) = \exp(\sin(\mathbf{X}_t))$$

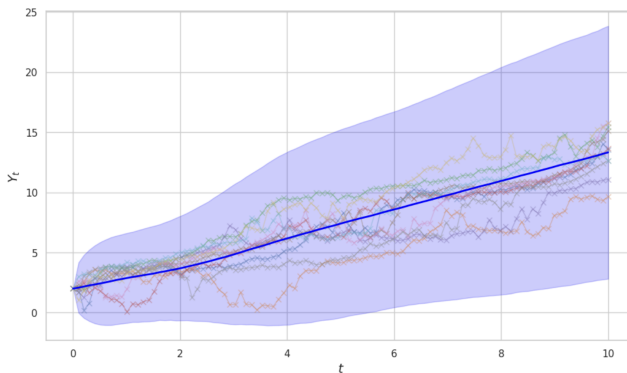


Figure: Under this setting, the mean curve increases stationary with t .

Complicate Function Experiment

$b(\mathbf{X}_t, t) = \exp(\cos(\mathbf{X}_t)), \sigma(\mathbf{X}_t, t) = \exp(\sin(\mathbf{X}_t))$				
d	I	b_{error}	D_{error}	b / D converge epochs
2	1000	0.286	0.0049	2000/2000
2(diff)	1000	0.0092	0.0245	500/500
5	1000	0.11	0.078	3000
5(diff)	1000	0.45	0.35	3000
5(diff)	2000	0.41	0.36	20000

Table: This table shows how the error varies when the functions become complicate under the MLE loss. As the dimension increases, both b and D error increase simultaneously significantly.

2D Experiment

We reconstruct the following 2D process, which is modified from a synthetic dataset:

$$d\mathbf{X}(t) = \mathbf{g}(\mathbf{X}(t))dt + \boldsymbol{\sigma}(\mathbf{X}(t))d\mathbf{W}_t, \mathbf{X}(t=0) = \mathbf{X}_0, t \in [0, T].$$

where $\mathbf{X}(t) = (X_1(t), X_2(t))^T \in \mathbb{R}^2$, $\mathbf{g}(\mathbf{X}) : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ is the drift function, and $\boldsymbol{\sigma} : \mathbb{R}^2 \rightarrow \mathbb{R}^{2 \times 2}$ is the diffusion function. The drift function $\mathbf{g}$ is given by[1]

$$\mathbf{g}(\mathbf{X}) = \left(\frac{1}{\sigma_1} \frac{N_1}{N_1 + N_2} (X_1 - \mu_{11}) + \frac{1}{\sigma_2} \frac{N_2}{N_1 + N_2} (X_1 - \mu_{21}), \right.$$

$$\left. \frac{1}{\sigma_1} \frac{N_1}{N_1 + N_2} (X_2 - \mu_{12}) + \frac{1}{\sigma_2} \frac{N_2}{N_1 + N_2} (X_2 - \mu_{22}) \right)^T,$$

$$N_1(\mathbf{X}) = \frac{1}{\sqrt{2\pi}\sigma_1} \exp\left(-\frac{(X_1 - \mu_{11})^2}{2\sigma_1^2} - \frac{(X_2 - \mu_{12})^2}{2\sigma_1^2}\right)$$

$$N_2(\mathbf{X}) = \frac{1}{\sqrt{2\pi}\sigma_2} \exp\left(-\frac{(X_1 - \mu_{21})^2}{2\sigma_2^2} - \frac{(X_2 - \mu_{22})^2}{2\sigma_2^2}\right).$$

The parameters are set as $\sigma_1 = 1, \sigma_2 = 0.95, \mu_{11} = 1.6, \mu_{12} = 1.2, \mu_{21} = 1.8, \mu_{22} = 1.0$. We set $T = 10.2$ and an initial condition $\mathbf{X}_0 = (2.0, 2.0)^T$ or $\mathbf{X}_0 \sim \mathcal{N}_2(0, \Sigma_0)$. We take the correlated diffusivity as

$$\boldsymbol{\sigma} = \begin{bmatrix} \sigma_0 \sqrt{|X_1|} & c_1 \sigma_0 \sqrt{|X_2|} \\ c_1 \sigma_0 \sqrt{|X_1|} & \sigma_0 \sqrt{|X_2|} \end{bmatrix}$$

2D Experiment

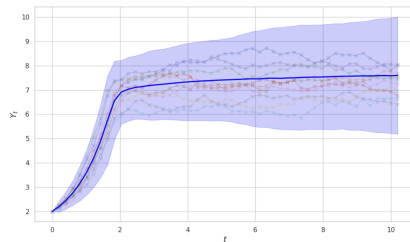
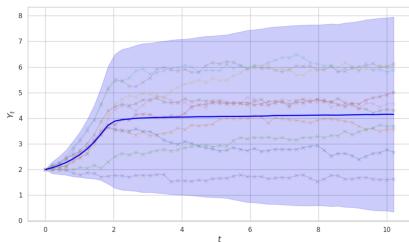


Figure: Trajectory examples, each sub-figure presents the projection onto the corresponding axis, shadows are drawn according to $\pm 3\sigma$

2D Experiment

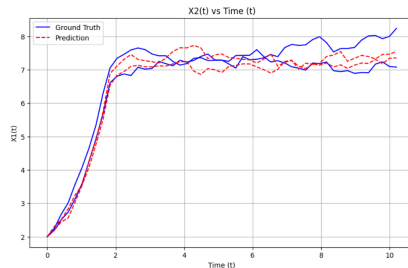
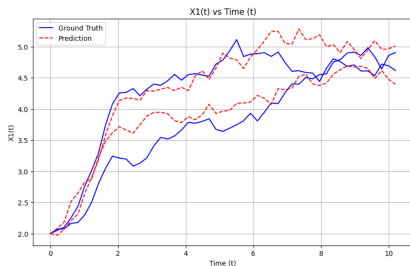


Figure: This figure shows trajectory reconstruction under MLE Loss, the related error of b and D are 0.049 and 0.037 respectively

Task Breakdown

1 Parameter Recovery

- Overview
- Diagonal Diffusion Coefficient
- Positive Defined Diffusion Coefficient

2 Solving PDEs with PINNs

- One-dimensional Burgers Equation
- 2D nonlinear Navier-Stokes equation

Tricks

- **PINN-w**: Larger weights are assigned to boundary conditions compared to other losses.
- **PINN-LRA (Loss Re-weighting with Annealing)**: This trick involves setting a weight updating factor α to adjust the loss weights during training.
- **RAR (Residual-based Adaptive Refinement)**: Refines the training process by adding new points where the residual is greatest every specific epochs.
- **gPINN (Gradient-enhanced PINN)**: A weight is assigned to the gradient terms (representing derivatives), while another weight is assigned to all other terms.

One-dimensional Burgers Equation

Given the Burgers' Equation:

$$u_t + uu_x = \nu u_{xx},$$

The domain is defined as

$$(x, t) \in \Omega = [-1, 1] \times [0, 1].$$

The initial and boundary conditions are

$$u(x, 0) = -\sin \pi x,$$

$$u(-1, t) = u(1, t) = 0.$$

The parameter is

$$\nu = \frac{0.01}{\pi}.$$

Hands On

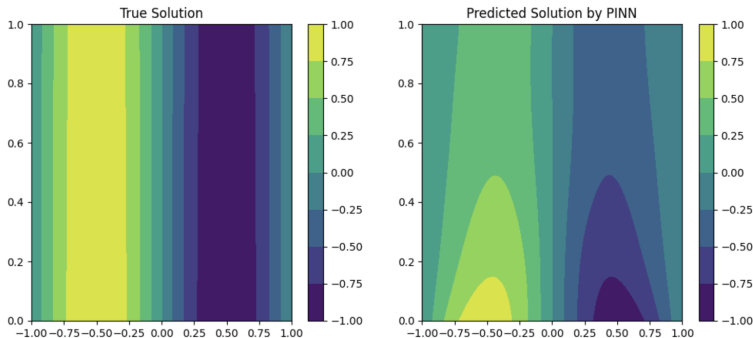


Figure: We consider construct the function and adjust the best weight of each item in loss function, the points are sampled uniformly.

PINNs-torch experiment

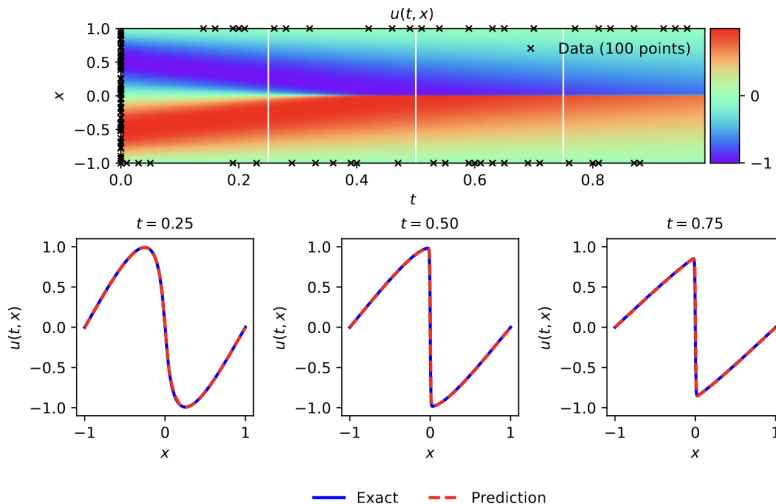


Figure: error is 0.0008, 0.0014 and 0.0029 when t is 0.25, 0.5 and 0.75, respectively.

2D nonlinear Navier-Stokes equation

Given the 2D nonlinear Navier-Stokes equation:

$$\begin{aligned}u_t + \lambda_1(uu_x + vu_y) &= -p_x + \lambda_2(u_{xx} + u_{yy}), \\v_t + \lambda_1(uv_x + vv_y) &= -p_y + \lambda_2(v_{xx} + v_{yy}),\end{aligned}$$

where $u(t, x, y)$ and $v(t, x, y)$ are the x and y components of the velocity field, and $p(t, x, y)$ is the pressure, we seek the unknowns $\lambda = (\lambda_1, \lambda_2)$.

When required, we integrate the constraints:

$$0 = u_x + v_y, u = \psi_y, v = -\psi_x,$$

We use a dual-output neural network to approximate $[\psi(t, x, y), p(t, x, y)]$, leading to a physics-informed neural network $[f(t, x, y), g(t, x, y)]$.

PINNs-torch experiment

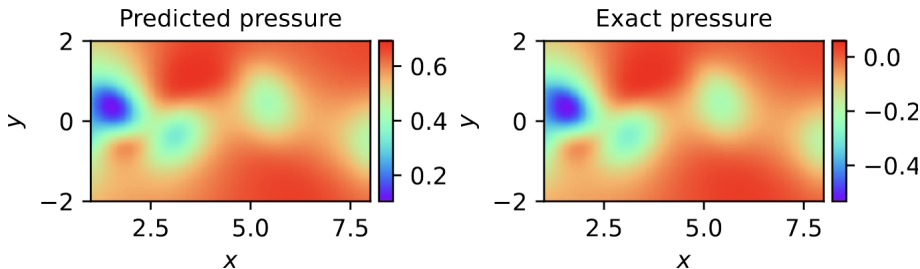


Figure: p appears only as a single derivative term, so the translation of the constant term is admissible. When the solution is normalized to mean 0, the relative error is 0.0079.

References I



S. MA, S. LIU, H. ZHA, AND H. ZHOU, *Learning stochastic behaviour from aggregate data*, in Proceedings of the 38th International Conference on Machine Learning, M. Meila and T. Zhang, eds., vol. 139 of Proceedings of Machine Learning Research, PMLR, 18–24 Jul 2021, pp. 7258–7267.

Thank you for listening!